



MovieVault

Project documentation and engineering handoff

MovieVault is a production-minded Expo mobile application for discovering movies across multiple industries using TMDb as the primary content source and OMDb for high-value enrichment such as IMDb ratings and box office information.

Project type: React Native mobile app built with Expo SDK 54

Primary platforms: Android-first Expo Go development flow with EAS-ready build setup

Data providers: TMDb for discovery, credits, providers, and imagery; OMDb for IMDb rating and box office

App package: com.movievault.app

Version: 1.0.0

Current workspace: F:\Movie_Vault

Documentation intent: this document captures the current implementation, technical architecture, product decisions, deployment setup, and recommended next steps for MovieVault as it exists in the codebase on April 27, 2026.

1. Product Overview

MovieVault is designed as a search-first movie information experience that brings discovery, industry browsing, cast navigation, and movie detail enrichment into a single mobile app.

The current product direction emphasizes speed, clean mobile presentation, and real metadata rather than placeholder content.

The home experience is curated around trending rails and industry-specific sections such as Hollywood, Bollywood, Tollywood, Kollywood, and Mollywood.

Movie detail pages enrich a title with ratings, runtime, cast, streaming providers, and production companies.

Actor pages surface profile information and filmography using both cast and crew credits, which is important for directors and writers such as Trivikram Srinivas.

2. Core Features

Home: Trending rail, multi-industry rails, themed hero area, and a search entry point that keeps discovery lightweight.

Search: Debounced global search, recent search history, and infinite query pagination for broader result coverage.

Industry pages: Dedicated wood screens with limited initial load and a wider industry search when the user starts typing.

Movie detail: Poster or backdrop hero, runtime, ratings, production, streaming providers, top cast, and box office enrichment.

Actor detail: Personal profile, biography, debut movie, top loaded box office title, and 3-column filmography.

Theme support: Dark and light modes with custom home interactions and screen-level theme treatment.

3. Technology Stack

Framework: Expo SDK 54 with React Native 0.81.5 and React 19.1.0.

Navigation: Expo Router using the file-based app directory.

Data and caching: TanStack Query 5 with custom service-layer caching and local persistence patterns.

UI utilities: Expo Image, Expo Linear Gradient, FlashList, AsyncStorage, and icon-driven components.

Build and release: Expo App Services configuration present in eas.json with development, preview, and production profiles.

4. Screen Inventory

app/index.tsx: Home screen with themed hero, animated top element, rails, and tap-through search bar.

app/search.tsx: Global search experience with recent history and paginated results.

app/category/[industry].tsx: Industry-specific catalog page with scoped search.

app/movie/[id].tsx: Movie detail page with providers above cast and a production section.

app/actor/[id].tsx: Actor and crew profile page with filmography.

app/watchlist.tsx: Watchlist route shell for future persistent user flows.

Current product note: the watchlist route exists in the navigation shell, but it should still be treated as an expansion path rather than a fully mature synchronized feature.

5. Data Model and API Flow

MovieVault uses TMDb as the authoritative source for discovery, credits, provider availability, imagery, and most detail-page metadata.

OMDb is called selectively for enrichment, mainly to surface IMDb rating and box office values where available.

The project normalizes remote responses into stable internal models: `MovieSummary`, `MovieDetail`, `PersonSummary`, and `PersonDetail`.

For actor pages, cast and crew movie credits are merged and deduplicated before rendering filmography so creators outside acting roles are still represented correctly.

MovieSummary: Lightweight card model used across home, search, and industry rails.

MovieDetail: Expanded title model with runtime, IMDb rating, providers, production companies, and curated people rail.

PersonSummary: Compact cast or crew representation used in movie detail.

PersonDetail: Profile, biography, debut movie, top loaded box office title, and filmography grid.

6. Performance and UX Patterns

The app is intentionally structured to feel fast even when upstream APIs are variable.

Debounced search reduces API churn and improves perceived responsiveness.

Parallel detail fetches reduce waterfalls on movie pages.

Home and category pages avoid loading overly large payloads up front.

Caching is used in both TanStack Query and service helpers to preserve speed and reduce repeat network work.

Search history is persisted locally for quick repeat actions.

Skeleton-first and themed UI decisions reduce the feeling of blank-state latency.

7. Project Structure

app: Route-level screens including home, search, category, movie, actor, and watchlist.

components: Reusable cards, lists, filters, search elements, and UI primitives.

hooks: Query and interaction hooks such as useHomeFeed, useMovieSearch, useMovieDetail, and search history.

services: TMDb client, OMDb client, normalization, HTTP helpers, and movie orchestration.

types: Shared TypeScript contracts for movies and people.

theme: Color tokens and theme-aware UI surface definitions.

8. Build and Deployment

Local development is currently optimized for Expo Go on Android. The application is configured to use automatic theme handling and is already wired with Expo Router and EAS metadata.

Recommended developer flow: 1) `cd F:\Movie_Vault` 2) `npm.cmd install` 3) `npm.cmd run start` 4) Open the QR session in Expo Go 5) Use EAS profiles defined in `eas.json` for builds.

Preview build intent: Internal Android testing and shareable install flow.

Production build intent: Release-ready store artifact pipeline.

Current icon and splash: Configured from `assets/movievault-logo.png` in `app.json`.

User interface style: automatic.

9. Security and Operational Notes

The current `app.json` stores TMDb and OMDb keys in `expo.extra`, which is acceptable for local prototyping but not ideal for a production-grade public client.

The architecture notes already point toward the correct next move: place API access behind a Node service with Redis caching.

That backend layer would improve key protection, cache sharing, rate-limit resilience, and normalization consistency.

Streaming provider availability is region-sensitive and depends on TMDb watch provider coverage.

10. Recommended Next Steps

Move API traffic behind a backend aggregator with Redis.

Implement a real persisted watchlist and optional authentication.

Persist TanStack Query cache for stronger offline behavior.

Add instrumentation for search latency, cache hit rate, and transition time.

Create a final build pipeline with tested icons, splash assets, and Play Store-ready metadata.

Bottom line: MovieVault already has the skeleton of a serious movie discovery product with normalized data, themed mobile UI, search-first flows, industry-specific browsing, and a codebase arranged for future scale. The biggest remaining step is moving from direct-client API usage to a production backend layer.